

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по лабораторной работе №1

Выполнил:
Студент группы ИУ5-35Б
Эльсон Э.К.
Преподаватель:
Гапанюк Ю. Е.

Москва 2025

Условия рубежного контроля №2 по курсу Пик ЯП

Рубежный контроль представляет собой разработку тестов на языке Python.

- 1) Проведите рефакторинг текста программы рубежного контроля №1 таким образом, чтобы он был пригоден для модульного тестирования.
- 2) Для текста программы рубежного контроля №1 создайте модульные тесты с применением TDD - фреймворка (3 теста).

Листинг кода

main.py

```
from operator import itemgetter
```

```
class StudentGroup:
```

```
    def __init__(self, id, name, student_count, course_id):
```

```
        self.id = id
```

```
        self.name = name
```

```
        self.student_count = student_count
```

```
        self.course_id = course_id
```

```
class Course:
```

```
    def __init__(self, id, name):
```

```
        self.id = id
```

```
        self.name = name
```

```
class GroupCourse:
```

```
    def __init__(self, course_id, group_id):
```

```
        self.course_id = course_id
```

```
        self.group_id = group_id
```

```

class DataService:

    def __init__(self, courses, groups, groups_courses):
        self.courses = courses
        self.groups = groups
        self.groups_courses = groups_courses

    def get_one_to_many(self):
        return [(g.name, g.student_count, c.name)
                for c in self.courses
                for g in self.groups
                if g.course_id == c.id]

    def get_many_to_many(self):
        many_to_many_temp = [(c.name, gc.course_id, gc.group_id)
                              for c in self.courses
                              for gc in self.groups_courses
                              if c.id == gc.course_id]

        return [(g.name, course_name)
                for course_name, course_id, group_id in many_to_many_temp
                for g in self.groups if g.id == group_id]

class GroupProcessor:

    def __init__(self, data_service):
        self.data_service = data_service

    def get_groups_with_courses_starting_with_a(self):
        one_to_many = self.data_service.get_one_to_many()
        res = list(filter(lambda i: i[2].startswith('A'), one_to_many))

```

```
return sorted(res, key=itemgetter(2))
```

```
def get_max_students_per_course(self):
```

```
    one_to_many = self.data_service.get_one_to_many()
```

```
    res_unsorted = []
```

```
    for c in self.data_service.courses:
```

```
        c_groups = list(filter(lambda i: i[2] == c.name, one_to_many))
```

```
        if len(c_groups) > 0:
```

```
            c_students = [student_count for _, student_count, _ in c_groups]
```

```
            c_max_students = max(c_students)
```

```
            res_unsorted.append((c.name, c_max_students))
```

```
    return sorted(res_unsorted, key=itemgetter(1), reverse=True)
```

```
def get_all_groups_with_courses_sorted(self):
```

```
    many_to_many = self.data_service.get_many_to_many()
```

```
    return sorted(many_to_many, key=itemgetter(1))
```

```
def main():
```

```
    courses = [
```

```
        Course(1, 'Архитектура АСОИУ'),
```

```
        Course(2, 'Аналитическая геометрия'),
```

```
        Course(3, 'Физика'),
```

```
        Course(4, 'Английский язык'),
```

```
        Course(5, 'Программирование'),
```

```
    ]
```

```
    groups = [
```

```
StudentGroup(1, 'ИУ5-35Б', 25, 1),
StudentGroup(2, 'ИУ6-51', 30, 2),
StudentGroup(3, 'МТ8-32Б', 20, 3),
StudentGroup(4, 'СМ2-15', 15, 4),
StudentGroup(5, 'РК1-66Б', 28, 1),
]
```

```
groups_courses = [
    GroupCourse(1, 1),
    GroupCourse(2, 2),
    GroupCourse(3, 3),
    GroupCourse(4, 4),
    GroupCourse(1, 5),
    GroupCourse(4, 1),
    GroupCourse(5, 2),
    GroupCourse(5, 5),
]
```

```
data_service = DataService(courses, groups, groups_courses)
processor = GroupProcessor(data_service)
```

```
print('Задание Г1')
res_1 = processor.get_groups_with_courses_starting_with_a()
for group_name, student_count, course_name in res_1:
    print(f'Курс: {course_name}, Группа: {group_name}, Студентов:
{student_count}')
```

```
print('\nЗадание Г2')
res_2 = processor.get_max_students_per_course()
```

```

for course_name, max_students in res_2:
    print(f'Курс: {course_name}, Макс. студентов в группе: {max_students}')

print('\nЗадание Г3')
res_3 = processor.get_all_groups_with_courses_sorted()
for group_name, course_name in res_3:
    print(f'Курс: {course_name}, Группа: {group_name}')

if __name__ == '__main__':
    main()

```

test_main.py

```

import unittest

from main import StudentGroup, Course, GroupCourse, DataService,
GroupProcessor

class TestGroupProcessor(unittest.TestCase):

    def setUp(self):
        self.courses = [
            Course(1, 'Архитектура АСОИУ'),
            Course(2, 'Аналитическая геометрия'),
            Course(3, 'Физика'),
            Course(4, 'Английский язык'),
            Course(5, 'Программирование'),
        ]

        self.groups = [
            StudentGroup(1, 'ИУ5-35Б', 25, 1),

```

```
StudentGroup(2, 'IY6-51', 30, 2),
StudentGroup(3, 'MT8-32B', 20, 3),
StudentGroup(4, 'CM2-15', 15, 4),
StudentGroup(5, 'PK1-66B', 28, 1),
]
```

```
self.groups_courses = [
    GroupCourse(1, 1),
    GroupCourse(2, 2),
    GroupCourse(3, 3),
    GroupCourse(4, 4),
    GroupCourse(1, 5),
    GroupCourse(4, 1),
    GroupCourse(5, 2),
    GroupCourse(5, 5),
]
```

```
self.data_service = DataService(self.courses, self.groups, self.groups_courses)
self.processor = GroupProcessor(self.data_service)
```

```
def test_get_groups_with_courses_starting_with_a(self):
    result = self.processor.get_groups_with_courses_starting_with_a()

    self.assertGreater(len(result), 0)

    for group_name, student_count, course_name in result:
        self.assertTrue(course_name.startswith('A'))

    sorted_result = sorted(result, key=lambda x: x[2])
```

```
self.assertEqual(result, sorted_result)
```

```
expected_courses = ['Аналитическая геометрия', 'Архитектура АСОИУ',  
'Английский язык']
```

```
result_courses = [course_name for _, _, course_name in result]
```

```
for expected in expected_courses:
```

```
    self.assertIn(expected, result_courses)
```

```
def test_get_max_students_per_course(self):
```

```
    result = self.processor.get_max_students_per_course()
```

```
    self.assertGreater(len(result), 0)
```

```
    for i in range(len(result) - 1):
```

```
        self.assertGreaterEqual(result[i][1], result[i + 1][1])
```

```
    result_dict = dict(result)
```

```
    self.assertEqual(result_dict.get('Архитектура АСОИУ'), 28)
```

```
    self.assertEqual(result_dict.get('Аналитическая геометрия'), 30)
```

```
    for course_name, max_students in result:
```

```
        self.assertIsInstance(max_students, int)
```

```
        self.assertGreater(max_students, 0)
```

```
def test_get_all_groups_with_courses_sorted(self):
```

```
    result = self.processor.get_all_groups_with_courses_sorted()
```

```
    self.assertGreater(len(result), 0)
```



```
sorted_result = sorted(result, key=lambda x: x[1])
self.assertEqual(result, sorted_result)

for group_name, course_name in result:
    self.assertIsInstance(group_name, str)
    self.assertIsInstance(course_name, str)
    self.assertGreater(len(group_name), 0)
    self.assertGreater(len(course_name), 0)

group_courses = [course_name for group_name, course_name in result
                  if group_name == 'ИУ5-35Б']
expected_courses_for_group = ['Архитектура АСОИУ', 'Английский язык']
for expected in expected_courses_for_group:
    self.assertIn(expected, group_courses)

if __name__ == '__main__':
    unittest.main(verbosity=2)
```

Результат выполнения программы:

```
PS C:\Users\elson\OneDrive\Рабочий стол\Лабы ПИКЯП\rk2> py .\main.py
```

Задание Г1

Курс: Аналитическая геометрия, Группа: ИУ6-51, Студентов: 30

Курс: Английский язык, Группа: СМ2-15, Студентов: 15

Курс: Архитектура АСОИУ, Группа: ИУ5-35Б, Студентов: 25

Курс: Архитектура АСОИУ, Группа: РК1-66Б, Студентов: 28

Задание Г2

Курс: Аналитическая геометрия, Макс. студентов в группе: 30

Курс: Архитектура АСОИУ, Макс. студентов в группе: 28

Курс: Физика, Макс. студентов в группе: 20

Курс: Английский язык, Макс. студентов в группе: 15

Задание Г3

Курс: Аналитическая геометрия, Группа: ИУ6-51

Курс: Английский язык, Группа: СМ2-15

Курс: Английский язык, Группа: ИУ5-35Б

Курс: Архитектура АСОИУ, Группа: ИУ5-35Б

Курс: Архитектура АСОИУ, Группа: РК1-66Б

Курс: Программирование, Группа: ИУ6-51

Курс: Программирование, Группа: РК1-66Б

Курс: Физика, Группа: МТ8-32Б

```
PS C:\Users\elson\OneDrive\Рабочий стол\Лабы ПИКЯП\rk2> █
```

```
PS C:\Users\elson\OneDrive\Рабочий стол\Лабы ПИКЯП\rk2> py .\test_main.py
```

```
test_get_all_groups_with_courses_sorted (__main__.TestGroupProcessor.test_get_all_groups_with_courses_sorted) ... ok
```

```
test_get_groups_with_courses_starting_with_a (__main__.TestGroupProcessor.test_get_groups_with_courses_starting_with_a) ... ok
```

```
test_get_max_students_per_course (__main__.TestGroupProcessor.test_get_max_students_per_course) ... ok
```

```
-----  
Ran 3 tests in 0.001s
```

OK

```
PS C:\Users\elson\OneDrive\Рабочий стол\Лабы ПИКЯП\rk2> █
```